



ROBOCUPJUNIOR RESCUE (LINE/MAZE) 2026

TEAM DESCRIPTION PAPER

Rescue Rats

Abstract

Our robot is a fully autonomous maze-navigating robot designed to locate victims and deploy rescue kits without any human input during a run. It is built around a dual-controller architecture, combining a Raspberry Pi 5 for high-level decision-making with an ESP32-based microcontroller for precise low-level hardware control. The two controllers communicate in real time over a UART serial connection at 115200 baud, with the Raspberry Pi 5 sending movement commands and the ESP32 returning sensor data and execution confirmations.

The chassis was fully designed in-house using 3D modeling software and printed by our team, giving us complete control over the layout, weight distribution, and component placement. Navigation relies on eight VL53L1X laser distance sensors covering all four directions, and an BNO055 gyroscope which tracks heading and actively corrects drift during straight movement. A TCS34725 color sensor on the underside detects floor tile colors to trigger rescue kit deployments automatically. Victim detection runs on the Raspberry Pi 5 using OpenCV, identifying colored ring targets by analyzing concentric color rings and recognizing letter victims by comparing contour shapes against pre-saved references for the Omega, Phi, and Psi symbols.

What sets our robot apart is the combination of a fully custom mechanical design, a clean split between high-level and low-level control, and a two-stage computer vision pipeline running entirely on an embedded system. Every part of the robot was built, programmed, and tested by our four-member team.

1. Introduction

Team

We are a team of four students who share a passion for robotics and technology. This year we took on the RoboCupJunior Rescue Maze challenge, building a fully autonomous robot that can navigate a maze, find victims, and deploy rescue kits on its own. It has been a challenging and rewarding experience that pushed each of us to grow, learn new skills, and work together toward a common goal.

Martin - Software

Martin handled all of the robot's programming, including maze-solving logic, sensor processing, and victim detection using a camera and OpenCV. With prior experience in Python and embedded C++, he built the entire software stack from scratch and was the go-to person whenever the robot wasn't behaving as expected.

Anton— Hardware & Electronics

Anton was responsible for building and wiring all of the robot's electronics, including the distance sensors, color sensor, and IMU. He had some background in electronics from previous projects and made sure every component was properly connected, tested, and working reliably throughout the competition season.

Karolína— 3D Modeling & Mechanical Design

Karolína designed and printed all the physical parts of the robot, from the main chassis to the rescue kit mechanism. With experience in 3D modeling software, she went through multiple design iterations to get everything fitting together just right.

Nikita— Poster & Bill of Materials

Nikita took care of all the team's competition poster, and the Bill of Materials. She made sure our work was clearly recorded and presented throughout the project, which was especially valuable during the busier phases when the rest of the team was deep in building and testing.

2. Project Planning

a. Overall Project Plan

Requirements and Constraints

- × We split the project into four main phases: design, build, software development, and testing. Since the mechanical structure needed to exist before electronics could be mounted, and electronics needed to be in place before software could be properly tested, we followed a sequential approach for the core work, while documentation and 3D modeling ran alongside the other phases throughout.

We held regular team meetings to check in on progress, discuss problems, and agree on next steps. Decisions were made as a group, and if someone was stuck, the rest of the team helped out.

b. Integration Plan

The integration of our robot followed a structured, phase-based approach driven by the hardware dependencies between subsystems. Because the mechanical frame had to be physically complete before any electronics could be mounted, and the electronics had to be in place before software could be meaningfully tested, we adopted a sequential core workflow while allowing parallel progress on 3D modeling, documentation, and software prototyping.

Phase 1 – Mechanical Assembly The robot's chassis was designed in-house using 3D modeling software and printed iteratively by our team. Once the base structure was finalized, the foldable walls were assembled and secured with screws. All mounting points for electronics were verified at this stage before any components were attached.

Phase 2 – Electronics Integration With the chassis ready, the electronic subsystems were installed and interconnected. All sensors — eight VL53L1X time-of-flight sensors, a BNO055 IMU, and a TCS34725 color sensor — were connected to the ESP32-S3 over a shared I2C bus. To avoid address conflicts among the VL53L1X sensors, each was individually initialized by toggling its XSHUT pin to assign a unique I2C address at startup. Separate screw-terminal buses were used for VCC (+5V) and GND lines to improve reliability over the earlier pin-type design. The Raspberry Pi 5 was mounted separately and connected to the ESP32 via UART serial at 115200 baud.

The power subsystem was wired with a 14.7 V LiPO battery feeding four JGY-370 motors through two motor drivers, and a step-down BUCK converter supplying 5 V to all logic-level components. PWM levels were tuned with a multimeter to keep motor temperatures within safe limits.

Phase 3 – Software Integration High-level software on the Raspberry Pi 5 was developed in Python as five separate modules: `main_controller.py`, `robot_movement.py`, `maze_map.py`, `victim_recognition.py`, and `utilities.py`. Low-level firmware on the ESP32 was written in C++ for Arduino as `robot_controller.ino`. The two layers communicate through a custom string-based protocol — the Raspberry Pi sends structured commands in `COMMAND:VALUE` format and blocks until it receives a `DONE` or `DONE:VALUE` response from the ESP32. Integration between the layers was validated by running commands through

the serial interface and confirming sensor feedback and motor responses. Emergency stop behavior was tested separately: when the physical switch triggers a STOP signal, the Raspberry Pi raises an EmergencyStopError and halts the current command immediately.

Phase 4 – System-Level Testing Once all three layers were integrated, the complete system was tested as a unit. Dedicated calibration sketches (pin_check.ino, variables_calibration.ino) were used to verify sensor addresses, motor directions, and encoder timing. Vision pipeline accuracy was validated using test_victim_recognition.py, which allowed us to tune BGR thresholds and observe ring classification in real time. After calibration, the robot consistently stopped within 2 cm of the target distance and within 2 degrees of the target heading. Throughout integration, team members coordinated closely — hardware and software issues that arose during combined testing were resolved collaboratively, with each person contributing based on their role.

- Robot systems architecture:



3. Hardware

- The whole electronic hardware can be separated into 2 distinct parts. Power (Motors) and Control (EP32, Raspberry PI, sensors, etc.) As for the non-electrical hardware, it's a 3D printed base with foldable 3D printed walls held in upright position by 4 screws.
- The Power part is controlled by the Control part via the ESP32 – S3 and a motor driver. The sensors use an I2C bus to communicate with the ESP. Moving distance is counted via the built-in encoders on the motors.

a. Mechanical Design and Manufacturing

The robot's structure consists of a custom 3D-printed base with foldable 3D-printed walls secured in an upright position by four screws. Designed by Karolína using 3D modeling software, the chassis went through multiple iterations to achieve optimal weight distribution and component placement. The entire exterior is printed with a textured matte black finish to minimize stray light reflections and protect optical sensors from data distortion. The top lid features an integrated carrying handle for easy retrieval and a vertical dispensing tower for rescue kits.

The chassis features precise mounting points to securely host the electronic architecture:

- Eight VL53L1X laser distance sensors are positioned at the corners of the side walls to detect maze walls and assist with trajectory realignment.
- A TCS34725 RGB sensor is mounted on the front underside to identify special floor tiles, such as silver checkpoints.

- A BNO055 IMU (accelerometer and gyroscope) is hosted internally for basic rotation control, tracking 90-degree turns, and increasing motor output on inclines.

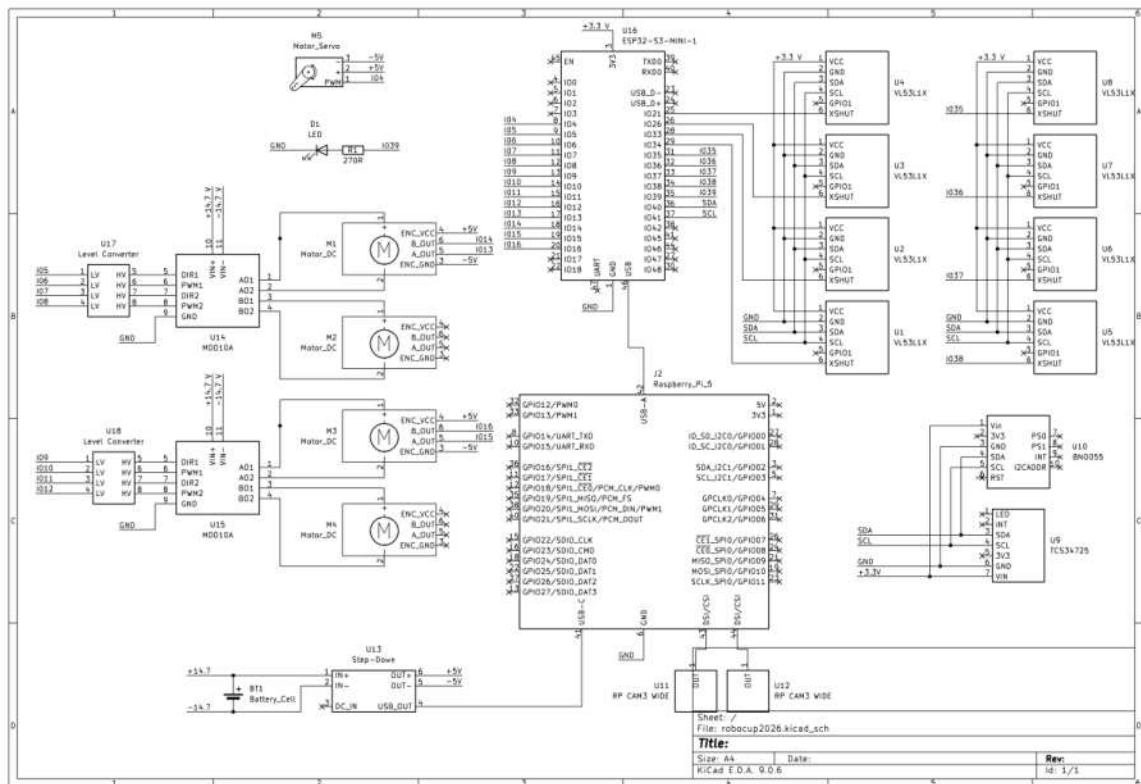
The rescue kit deployment mechanism is positioned at the top rear of the chassis. It consists of a vertical 3D-printed gravity-fed magazine tower and a 5V servo motor. When the high-level OpenCV vision pipeline or floor color sensor flags a victim tile, the servo actuates an internal shelf to drop exactly one rescue kit by gravity, reliably preventing double drops.



b. Electronic Design and Manufacturing

- All the sensors communicate with the ESP via an I2C bus, these sensors include:
 - VL53L1X – A time of flight laser sensor, appears 8 times inside the construction, one on each corner of the sidewall. They're used to help the robot react to the walls of the maze while also allowing for subsequent turning when out of alignment with the walls. With the number of sensors on the I2C, we are forced to make use of the XSHUT pins on the VL53L1X which allows us to overwrite the default I2C address, so the sensors don't try to allocate the same addresses.
 - BNO055– An accelerometer with a gyroscope and a geomagnetic sensor, appears once in the construction, used for basic rotation control, allowing the ESP to know if the motors made a 90° turn, while also allowing the motors to raise their output on a slope.
 - TCS34725 – An RGB sensor, located on the front side of the construction, used for recognizing special tiles on the ground.
- Main controller:

- The main controller used in the electrical part would be the ESP32, carrying code and both receiving information from the sensors, while also being responsible for controlling the motors through the motor drivers.
- Power subsystem
 - The power part is powered by a 14.7 V (actually around 15.17V) LiPO battery, while the control part is powered through a step-down BUCK converter (down to 5V). The 14.7V part consists of 4 motors and also 2 motor drivers, the motors although run at a lower PWM (around 60-80/255) to lower the voltage output thus preventing unnecessary overheating.
- Actuators, etc.:
 - For movement 4 JGY-370 motors(12V) with built in encoders are used for general movement, with a 5V servo motor used for dispensing rescue kits.



Schema of the internal electronic components made via KiCAD

- We used a multimeter while adjusting the PWM input in the motor drivers, changing the voltage and amperage consumption, therefore lowering the chances of overheating. Combined with constant testing and repairing electronics.
- To avoid excess soldering of the SDA and SCL cables onto one pin, we chose to go with an I2C bus, the first version had pin-type I2C bus, which was however replaced for its flimsiness and unreliability with a more sturdy and reliable screw-in terminal style bus. The same applies for the other buses which were used like the VCC (+5V) bus and the GND bus. And as to avoid interference from the Motor drivers we used to wrap some of the I2C cables with aluminum foil, for the same reason we also intertwined individual SDA and SCL cables, creating an enclosed electromagnetic field which helps with lowering the interference from outside sources.

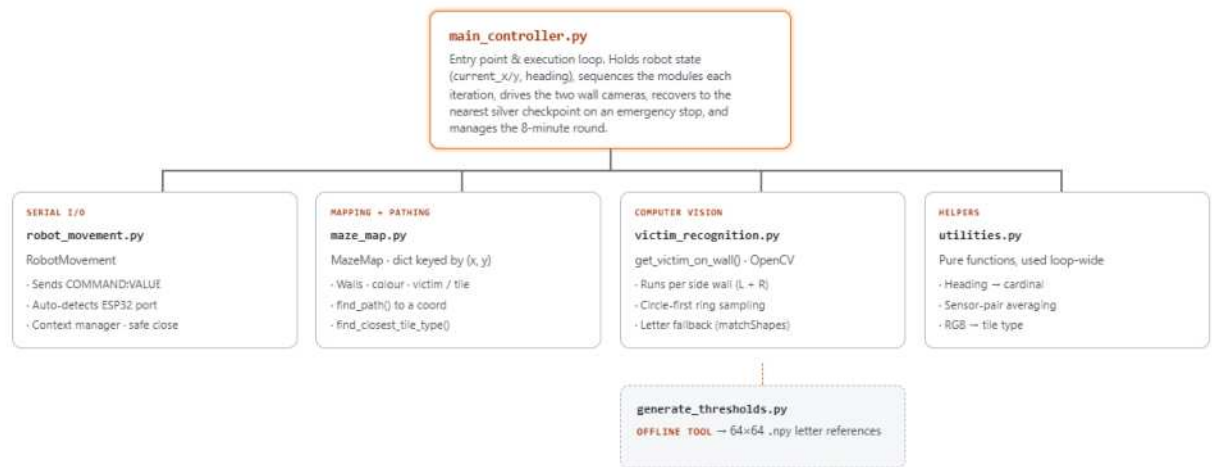
4. Software

- The robot's software is divided into two sections. High level control logic written in Python running on Raspberry Pi 5 and a low level control running on ESP32-S3 written in C++ for Arduino. Communication between these two sections is handled over serial connection. Connection runs by default on 115 200 baud rate, its uses custom string-based protocol.

a. General software architecture

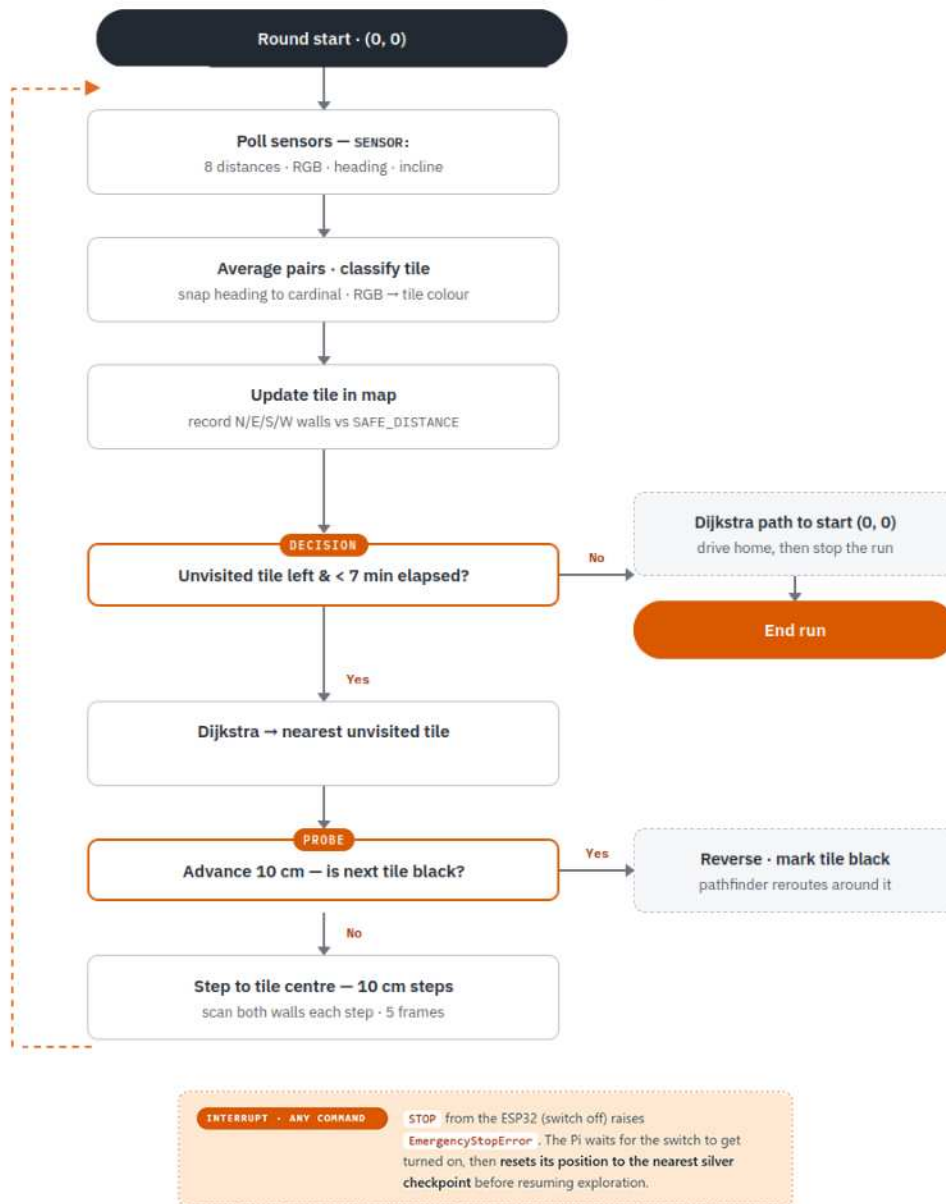
Raspberry Pi – High level control

- The Raspberry Pi software is made of 5 separate files, each handling its own part. The main logic loop and start-up point is `main_controller.py`. It integrates the other 4 files, handling them and making movement decisions. It tracks the robot's current location, heading. It also handles emergency stops: when `EmergencyStopError` is raised, `reset_to_nearest_checkpoint()` blocks until the physical switch is restored, then resets the tracked position to the nearest mapped silver tile (or the start tile if none exists), allowing the run to continue even after a lack of progress.



- `robot_movement.py` handles the serial communication. It sends structured commands to the ESP32 in `COMMAND:VALUE` format (`FORWARD:30`, `RIGHT:90`, `SENSOR:`). It blocks the execution loop until the ESP32 responds with `DONE` or `DONE:VALUE` where value can be a measured value such as actually traveled distance. At the initialization stage, on Linux, it tries to automatically connect to the ESP32 via the path `/dev/serial/by-id/usb-Silicon_Labs`. If the ESP32 sends a STOP signal (triggered by the physical switch), it raises an `EmergencyStopError` exception, which halts the current command immediately. It implements python's context manager to ensure that when the program ends or crashes the serial connection gets closed, as not to block any future connection.
- `maze_map.py` handles the mapping of the maze. It uses a dictionary where the key is a tuple of the tile coordinates. The value for each coordinate is a dictionary consisting of: a `walls` dictionary mapping whether there is a wall in each cardinal direction, `tile color` as a string describing the tile's type and a `victim` saving victim value on each tile to prevent double drops of med kits. The class includes multiple functions to edit each tile's attributes as well a way to get each tile's attribute. It also exposes 2 path-finding functions based on Dijkstra's algorithm, where one returns a path to specific coordinate and one that returns path to the closest specified tile type.
- `victim_recognition.py` uses OpenCV to identify victims on walls from camera frames. Firstly it edits the photo's contrast and saturation values. Then it thresholds the image. Calculating value of the circle if present if not then using OpenCV's built in `matchShapes` function to compare the frame to reference photos of the Greek letters. Greek letter references are pre-generated by `generate_thresholds.py`, which processes reference images and saves 64x64 image arrays in .npy format.
- `utilities.py` provides helper function to `main_controller.py`, such as converting number heading to cardinal directions, converting RGB values to color strings, averaging sensor distances.

- Each iterations flow:



ESP32 – low level control

- robot_controller.ino runs the hardware control loop. Parses incoming serial commands. Controls four DC motors, reads 8 VL53L1X laser distance sensors, a BNO055 IMU, and a TCS34725 color sensor. Movement uses encoder-based movement to move a set distance using a time-based control as fallback with active IMU heading correction to keep the robot moving straight. A function table (array of command-name/function pair) runs all recognized commands without if-else chains.

Tools and libraries

Platform	Libraries
Raspberry Pi	Pyserial, OpenCV-python, NumPy, Picamera2, Matplotlib
ESP32	VL53L1X by Pololu, Adafruit BNO055, Adafruit TCS34725, ESP32Servo

b. Innovative solutions

- The maze exploration uses a weighted version of Dijkstra's algorithm. Each tile type is given a cost: unvisited tiles cost 1, visited and silver tiles cost 2, incline tiles cost 3, red tiles cost 10, blue tiles cost 50, and black tiles cannot be entered at all. This means the robot prefers exploring new tiles over revisiting ones it has already seen, while still avoiding penalty zones. The path is recalculated at the start of each exploration cycle. Once a path is computed, the robot follows it to the target tile, if a wall blocks movement mid-path, the failed move is flagged and a new path is computed on the next cycle.
- The maze map class utilizes Python's context manager. Thanks to the context manager, no matter if the program crashes or ends, it will always export and save the map in .npy format.
- The serial communication file also utilizes Python's context manager. In this case, it guarantees that the serial connection gets closed, no matter if the program crashes or ends. Allowing for the serial connection to be established again in the next run.
- Victim detection works in two stages. First, the robot tries to identify a colored circle by sampling the average color of five rings around the largest shape in the frame. Each ring is scored based on its color. If all five rings are identified, their scores are added up and returned as the victim value. If any ring cannot be identified, the robot falls back to shape matching — comparing the frame against reference images of the three Greek letters using OpenCV's `matchShapes`.
- To avoid false detections from a single bad frame, victim recognition runs five times in a row for each wall. The result that appears most often across the five readings is used as the definitive answer.

Reliability Tests and Quality Assurance

- The `pin_check.ino` file is used to check whether all pin number are correct, it uses terminal approach iterating through each motor and distance sensor. Always prints out what will be ran, for distance sensor we put hand in front of the supposed sensor to check whether the distance gets smaller.
- The `variables_calibration.ino` also uses terminal approach where it moves for 2 seconds, measures distance travelled and calculates how long it takes to move one centimeter. Then it turns to the right for 2 second measuring how long it takes to rotate by one degree. Then it turns left, comparing whether the change in heading was positive or negative, to identify which rotation the IMU tracks as positive
- The `map_visualise.py` file is used to debug each runs map output. It uses matplotlib library to draw each tile and its type, all the walls and the victims.
- The `test_victim_recognition.py` file is used to calibrate the BGR values for each color on the victims. It draws original frame, colour edited frame, thresholded image and the debug image. On the debug image, we draw the biggest contour. Around that biggest contour gets drawn the 5 rings of cognitive victims. We write each rings colour classification as well its individual BGR values, we also draw similarity score for each letter victim.
- Sensor data is validated every read — if more than 2 of the 13 values come back as invalid, the read is retried up to 3 times before continuing.

- After calibration the robot consistently stopped within 2 cm of the target distance. It also stopped within 2 degrees of error while turning, which then got even lower by realigning itself based on distances to the nearest wall.

5. Performance evaluation

The robot's performance was evaluated through targeted unit tests, calibration routines, and full-run simulations on a test maze built to replicate competition conditions.

Movement Accuracy Motor calibration was performed using `variables_calibration.ino`, which measured the time required to travel one centimeter and rotate one degree. After tuning the encoder-based movement and IMU heading correction, the robot consistently stopped within 2 cm of the target distance. Rotational accuracy settled within 2 degrees of error, reduced further at runtime by realigning against the nearest wall using the VL53L1X sensors.

Sensor Reliability If more than 2 out of 13 sensor values returned as invalid in a single read cycle, the read was retried up to 3 times automatically. All eight VL53L1X sensors were verified individually using `pin_check.ino`, confirming a measurable distance change when an object was placed in front of each one.

Victim Detection The vision pipeline was evaluated using `test_victim_recognition.py`, which displayed the original, color-adjusted, and thresholded frames alongside a debug overlay. BGR thresholds were tuned until recognition was reliable across different lighting conditions. To reduce false positives, recognition ran five consecutive times per wall, with the most frequent result accepted as the final answer.

Maze Navigation The weighted Dijkstra pathfinding was validated by observing that the robot consistently explored unvisited tiles before revisiting known ones. Path recalculation at the start of each exploration cycle ensured correct adaptation when new walls were discovered mid-run.

Overall Assessment The robot performed reliably across all tested subsystems. The areas most sensitive to tuning were victim detection under variable lighting and wall alignment in narrow corridors, both of which were resolved through iterative testing before competition.

6. Conclusion

In this paper, we presented the comprehensive design and implementation of our autonomous maze-navigating robot for the RoboCupJunior Rescue Maze challenge . By leveraging a balanced dual-controller architecture that pairs a Raspberry Pi 5 for high-level decision-making with an ESP32-S3 for precise low-level hardware control, the system successfully accomplishes fully autonomous grid mapping, wall alignment, and victim identification . The custom, 3D-printed chassis with Mecanum wheel drive system provides optimal mobility and serviceability , while the multi-sensor array ensures reliable environmental feedback . Rigorous system calibration and testing routines—specifically through targeted scripts like `variables_calibration.ino`—validated that the robot consistently operates within tight tolerance thresholds, keeping displacement error below 2 cm and rotational variance within 2 degrees .

Innovative software strategies, such as weighted Dijkstra pathfinding and a fallback two-stage computer vision pipeline , combined with robust mechanical workarounds against electromagnetic interference , allowed us to meet all core project constraints. The iterative engineering process behind this development has significantly advanced our team's hands-on expertise in embedded programming, computer vision, electrical isolation, and collaborative systems integration .

References

<https://www.redblobgames.com/pathfinding/a-star/implementation.html#python-dijkstra>
<https://docs.python.org/3/>

<https://bradfieldcs.com/algorithms/graphs/dijkstras-algorithm/>

<https://matplotlib.org/stable/index.html>

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/>

<https://github.com/pololu/vl53l1x-arduino>

[https://opencv-](https://opencv-tutorial.readthedocs.io/en/latest/search.html?q=matchShapes&check_keywords=yes&area=default)

[tutorial.readthedocs.io/en/latest/search.html?q=matchShapes&check_keywords=yes&area=default](https://opencv-tutorial.readthedocs.io/en/latest/search.html?q=matchShapes&check_keywords=yes&area=default)

https://github.com/adafruit/Adafruit_BNO055

https://github.com/adafruit/Adafruit_TCS34725

<https://numpy.org/doc/2.4/>

<https://pyserial.readthedocs.io/en/latest/>

<https://pip-assets.raspberrypi.com/categories/652-raspberry-pi-camera-module-2/documents/RP-008156-DS-2-picamera2-manual.pdf>